

UNIVERSIDAD NACIONAL DE SAN AGUSTÍN

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



COMPUTACIÓN GRÁFICA, VISIÓN COMPUTACIONAL Y MULTIMEDIA

Grupo B

Docente: Rene Alonso Nieto Valencia

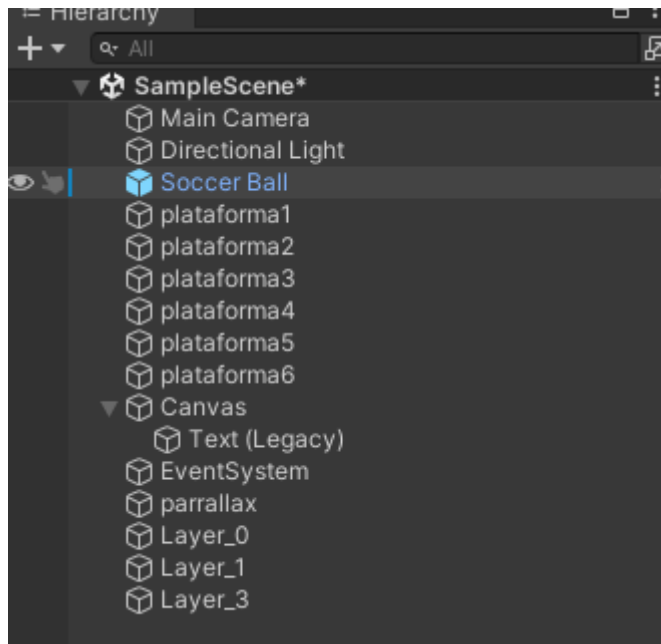
Estudiante: Jhoer Luis Vera Zapacayo

AREQUIPA – PERÚ
2026

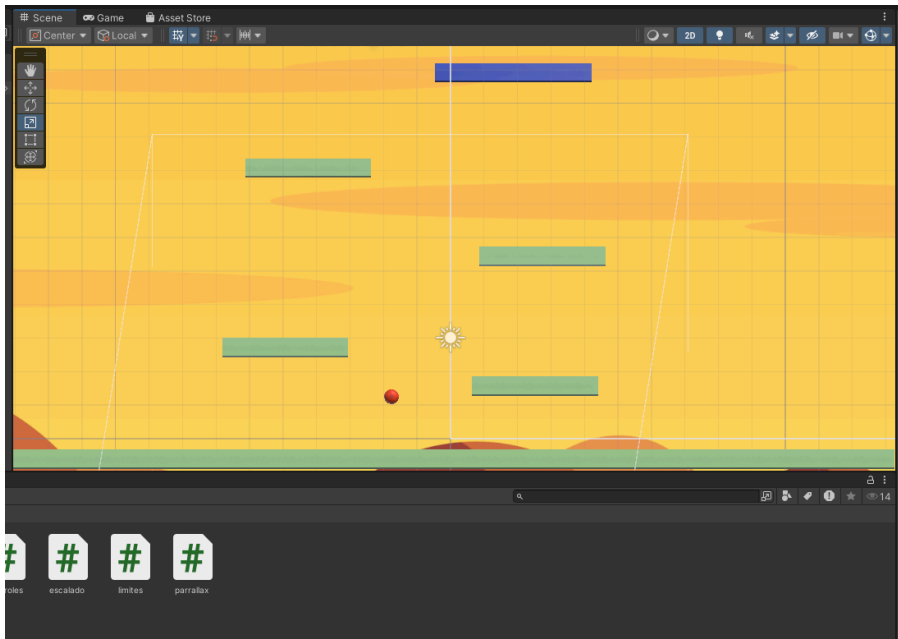
1. Descripción general de la aplicación

Este trabajo fue desarrollado en Unity utilizando un entorno de juego 2D. El personaje principal es un pelota que salta entre plataformas con el objetivo de llegar a una plataforma final . Se aplicaron las transformaciones geométricas de traslación, rotación y escalado, además del efecto visual parallax para el fondo, mejorando la experiencia visual del jugador.

Jerarquía creada para el Proyecto



Escena del Proyecto



2. Transformaciones geométricas utilizadas

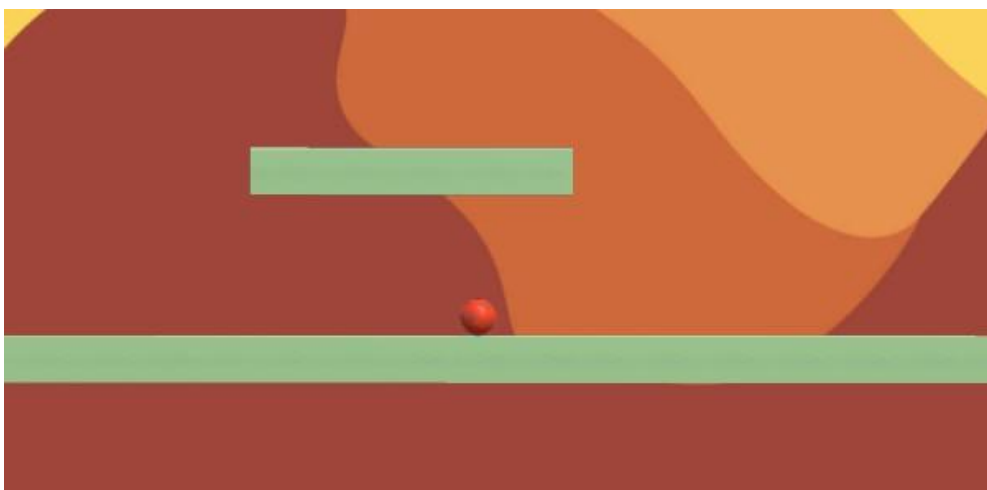
a. Traslación

Se aplicó traslación a la pelota mediante la fuerza horizontal:

```
CSharp
float movimiento = Input.GetAxis("Horizontal");

if (Mathf.Abs(rb.velocity.x) < maxVelocidad)
{
    rb.AddForce(new Vector3(movimiento * fuerzaMovimiento, 0, 0));
}
```

Esta lógica permite mover la pelota a la izquierda o derecha, desplazándose por el espacio del juego.



b. Rotación

El planeta rota suavemente mientras se mueve, dando una sensación de rodamiento:

```
CSharp
transform.Rotate(0, 0, -movimiento * 2f);
```

Esto mejora el realismo visual sin alterar la física del objeto.

c. Escalado

Las plataformas se pueden escalar con las teclas "T" (aumentar) y "U" (disminuir):

```
CSharp
if (Input.GetKeyDown(KeyCode.T))
{
    transform.localScale += new Vector3(1f, 0, 0);
}

if (Input.GetKeyDown(KeyCode.U))
{
    transform.localScale -= new Vector3(1f, 0, 0);
}
```

Esto permite modificar el tamaño de la plataforma durante la partida, afectando la dificultad.

3. Efecto Parallax

Para crear profundidad visual, se aplicó el efecto parallax en los fondos (como nubes o montañas):

```
CSharp
Vector3 movimiento = camara.position - ultimaPosCamara;

transform.position += new Vector3(
    movimiento.x * velocidadParallax,
    movimiento.y * velocidadParallax,
    0
);

ultimaPosCamara = camara.position;
```

Esto hace que los fondos se muevan más lentamente que la cámara, generando una sensación de desplazamiento en capas.

4. Lógica de reinicio al llegar a la plataforma final

Se detecta colisión con la plataforma final desde arriba para reiniciar el nivel:

```
CSharp
if (collision.gameObject.CompareTag("Final"))
{
    foreach (ContactPoint contacto in collision.contacts)
    {
        if (Vector3.Dot(contacto.normal, Vector3.up) > 0.5f)
        {
            transform.position = posicionInicial;
            rb.velocity = Vector3.zero;
            break;
        }
    }
}
```

5. Conclusiones

Este proyecto me permitió comprender e implementar conceptos clave de transformaciones geométricas en Unity, así como mejorar la experiencia visual mediante el uso del parallax. La combinación de estas técnicas resultó en una aplicación funcional, visualmente atractiva y jugable.

6. Bibliografía o recursos utilizados

- [1] <http://www.cs.uns.edu.ar/cg/clasespdf/2.1-Transformaciones.pdf>
- [2] https://www.youtube.com/watch?v=7wah5FNsz68&ab_channel=MasterHeHeGar
- [3] <https://overfilegames.com/2023/07/06/%F0%9F%8C%9F-crear-un-videojuego-de-plataformas-con-unity-%F0%9F%8E%AE-6-crear-efecto-parallax/>
- [4] <https://assetstore.unity.com/packages/tools/particles-effects/free-parallax-for-unity-2d-29422?srltid=AfmBOor>
- [5] <https://martra.uadla.com/como-crear-un-efecto-parallax-en-unity-de-una-forma-acojonantemente-sencilla>